

FBI-LLM: Scaling Up Fully Binarized LLMs from Scratch via Autoregressive Distillation

Liqun Ma¹, Mingjie Sun², Zhiqiang Shen¹

¹Mohamed bin Zayed University of AI

²Carnegie Mellon University

{Liqun.Ma,Zhiqiang.Shen}@mbzuai.ac.ae, mingjies@andrew.cmu.edu

Abstract

This work presents a **Fully Binarized Large Language Model (FBI-LLM)**, demonstrating for the first time how to train a large-scale binary language model from scratch (not the partial binary or ternary LLM like BitNet b1.58 [1]) to match the performance of its full-precision counterparts (e.g., FP16 or BF16) in transformer-based LLMs. It achieves this by employing an autoregressive distillation (AD) loss with maintaining equivalent model dimensions (130M, 1.3B, 7B) and training data volume as regular LLM pretraining, while delivering competitive results in terms of perplexity and task-specific effectiveness. Intriguingly, by analyzing the training trajectory, we find that the pretrained weight is not necessary for training binarized LLMs from scratch. This research encourages a new computational framework and may facilitate the future design of specialized hardware tailored for fully 1-bit LLMs. We make all our models, code, and training dataset fully accessible and transparent to support further research¹.

1 Introduction

Benefiting from the huge parameter scale and massive training corpora, transformer-based Large Language Models (LLMs), like ChatGPT [2] and LLaMA [3, 4], perform great in tasks requiring domain knowledge and complex reasoning. Moreover, the capabilities of LLMs tend to enhance as their parameter sizes expand. This substantial scale in parameters results in considerable storage and computational demands, which substantially restricts LLMs’ broader application and development. Quantization efficiently mitigates these limitations by mapping 32-bit parameters to smaller bit sizes. It substantially cuts storage requirements and enhances computational speed and energy efficiency during inference.

As the most extreme case of quantization, binarization represents each parameter by just $\{-1, 1\}$. It maximizes compression and efficiency but at the cost of accuracy. Prior efforts to preserve the efficacy of binarized LLMs include retaining salient parameters [5] or using near-one-bit to represent each parameter [1]. While these approaches have shown promise, they still leave room for optimization in storage and efficiency, and additional full-precision parameters or parameter encodings expressed in non-powers of 2 can cause extra overhead when adapting to edge hardware.

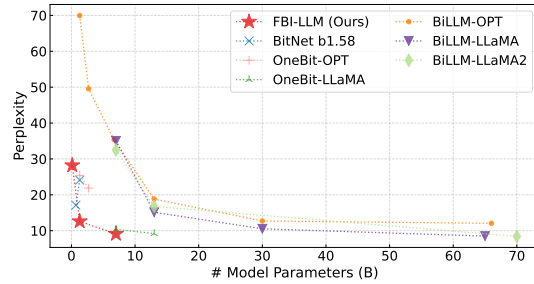


Figure 1: Perplexity on Wikitext2 of existing binarized LLMs and our FBI-LLMs. FBI-LLMs get similar or lower magnitude of perplexity on similar size of models compared with other binarized LLMs.

¹Code: <https://github.com/LiqunMa/FBI-LLM>. Model: <https://huggingface.co/LiqunMa/>.

Networks (BNNs) to improve their accuracy despite inherent limitations. BinaryConnect [8] converts full-precision weights in neural networks to 1-bit binary weights using stochastic methods during training and simulates the effects of binary weights during inference. They also implement a clipping function in backward propagation to prevent excessive growth of real-valued weights. Expanding on this, they develop the Binarized Neural Network (BNN) [9], which includes detailed training and acceleration techniques, demonstrating the efficiency and practicality of BNNs through reduced storage and faster processing times in image classification.

However, these methods typically suffer from accuracy loss, prompting numerous optimization-based solutions over recent years to mitigate this. Binary Weight Networks (BWN) and XNOR-Net [10] introduce a scaling factor that approximates floating-point parameters to reduce quantization errors. Further developments like DoReFa-Net [11], WRPN [12], and ABC-Net [13] introduce strategies to minimize information loss and quantization errors. Innovations such as XNOR-Net++ [10] and various mimic solutions like Distillation and Quantization (DQ) [14] continue to refine these approaches, emphasizing stability and high accuracy in training binary models. To address the non-differentiability of the binarization function, techniques like the straight-through estimator (STE) [15] are used for backpropagation. The ReActNet [16] improves BNNs with generalized activation functions, and the BNN+ [17] introduces an enhanced derivative approximation of the sign function along with a regularization strategy to optimize weight learning.

Large Language Model Binarization. PB-LLM [5] implements partial binarization of the LLMs, retaining the salient parameters at full precision, occupying only a small portion of all parameters, to maintain the linguistic reasoning capacity. BiLLM [18] also considers the distribution pattern of the weight scale. It uses a binary residual approximation strategy to binarize salient parameters, which consists of an original binary tensor and a residual binarized matrix to present the binarization result of salient parameters. BitNet b1.58 [1] quantizes all parameters to the set of $\{-1, 0, 1\}$, where they find that quantized LLMs achieve competitive performance to their full-precision counterparts. However, PB-LLM and BiLLM use the extra cost of storage to process salient weights, and BitNet b1.58 uses an average of 1.58 bits to represent weights. None of them have reached the limits of binary models. There is room for further improvement in model storage size and inference speed. Our work focuses on achieving fully binarized LLMs while preserving the model’s capabilities as much as possible.

BitNet [19] and OneBit [7] employ quantization-aware training (QAT) to binarize LLMs. BitNet utilizes group quantization by applying different scales to the parameters of various groups, which accelerates model training. Its training loss aligns with that of pretraining autoregressive language models. Conversely, OneBit preserves the full-precision model knowledge through quantization-aware knowledge distillation, using a full-precision model as the teacher and guiding the binarized model training with two distinct loss functions. Unlike these two methods, our approach achieves similar or better results through a more streamlined and efficient training process.

3 Methodology

In this section, we first provide an overview of the architecture of our FBI-LLM in Section 3.1. Then, in Section 3.2, we detail the FBI-linear module, the main component of FBI-LLM. Finally, we elaborate the FBI-LLM autoregressive distillation-based training procedure in Section 3.3.

3.1 Architecture of FBI-LLM

We illustrate the overall architecture of FBI-LLM in Fig. 2 (a). In transformer-based LLMs, the majority of parameters are found within the linear modules. FBI-LLM replaces all linear modules, except for the causal head, with FBI-linear (Fully Binarized Linear). Since the causal head directly influences the output token distribution in each step, binarizing its parameters would significantly affect the accuracy of the model’s output, so we retain its precision.

Additionally, the parameters in two other core modules of LLMs, embedding and layer norm, also need to be kept at full precision. This is because the embedding module contains semantic information about all tokens and, as the first layer of the model input, determines the text’s initial representation. Layernorm, on the other hand, scales the activation values directly. Binarizing its parameters would significantly reduce the semantic expressiveness of the activation values at each layer. Similar settings are also adopted in other work [19] about the binarization of LLMs.

3.2 FBI-Linear

The main parameters in FBI-linear are a matrix $\mathbf{W}^b \in \mathbb{R}^{m \times n}$ consisting only of $\{1, -1\}$, which is binarized from the full-precision linear module $\mathbf{W}^f \in \mathbb{R}^{m \times n}$ of the LLMs. During the training, the binarization process can be formulated as:

$$\mathbf{W}^b = \text{sign}(\mathbf{W}^f) \quad (1)$$

where sign function is formulated as:

$$\text{sign}(\mathbf{W}_{ij}^f) = \begin{cases} 1, & \mathbf{W}_{ij}^f > 0 \\ -1, & \mathbf{W}_{ij}^f \leq 0 \end{cases} \quad (2)$$

We follow the previous works [10, 19] to scale the binarized parameter with full-precision scale factors. Scale factors can effectively reduce the error between the binarized and original parameters, thereby preserving the more representational capacity of the corresponding module. They constitute a small fraction of the total parameters, making them a highly efficient enhancement to the model's performance without significantly increasing the parameter count and computational demand.

Specifically, in the FBI-linear, we apply scaling at the granularity of the matrix columns. The calculation process can be formulated as:

$$\widetilde{\mathbf{W}}_{:,j}^b = \alpha_j \mathbf{W}_{:,j}^b + \beta_j \quad (3)$$

where $\widetilde{\mathbf{W}}_{:,j}^b$ donate the j^{th} column of the scaled binarized weight matrix $\widetilde{\mathbf{W}}^b \in \mathbb{R}^{m \times n}$. α_j and β_j are the j^{th} element in learnable scale vectors $\boldsymbol{\alpha} \in \mathbb{R}^n$ and $\boldsymbol{\beta} \in \mathbb{R}^n$ respectively.

To accelerate the model's convergence speed, we initialize $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ before training as following:

$$\alpha_j = a_j \quad (4)$$

$$\beta_j = \frac{1}{m} \sum_i^m |\mathbf{W}_{ij}^f - a_j| \quad (5)$$

where $a_j = \frac{1}{m} \sum_i^m \mathbf{W}_{ij}^f$, denotes the average of j^{th} column in $\mathbf{W}^f$.

3.3 Autoregressive Distillation

Given a training corpus of tokens $\mathbf{x} = \{x_1, \dots, x_n\}$, a standard autoregressive language modeling objective [20] is to maximize the likelihood:

$$\mathcal{L}(\mathbf{x}) = \sum_i \log p(x_i | x_{i-k}, \dots, x_{i-1}; \boldsymbol{\theta}) \quad (6)$$

where k represents the size of the context window and the conditional probability p is modeled through a neural network characterized by the parameters $\boldsymbol{\theta}$. Unlike conventional autoregressive language models, we train FBI-LLM using **autoregressive distillation (AD)**. In the training, a full-precision pre-trained LLM is used as the teacher model, and the binarized target model acts as the student. Suppose each instance of training data consists of a sequence of input tokens $x^1, \dots, x^m$, the teacher prediction probability for the next token can be formulated as:

$$\mathbf{p}^T(x^{m+1} | x^1, \dots, x^m) = \text{softmax}(\mathbf{h}_l^m \mathbf{W}_{m+1}). \quad (7)$$

where $\mathbf{h}_l^m$ represents the activation of the final transformer block, $\mathbf{W}_{m+1}$ represents parameters of the added linear output layer to predict the next token's probability.

The cross-entropy between the outputs of the student model and the teacher model is calculated as the final loss function at each step of predicting the next token. It can be formulated as:

$$\mathcal{L} = -\frac{1}{n} \sum_i^n \mathbf{p}^T(x^{i+1}) \cdot \log \mathbf{p}^S(x^{i+1}) \quad (8)$$

Table 1: Hyper-parameters for FBI-LLMs in experiments.

Model Size	# layers	hidden size	# attention heads	intermediate size	batch size (tokens)
FBI-LLM 130M	12	768	12	2,048	2M
FBI-LLM 1.3B	24	2,048	32	5,632	2.4M
FBI-LLM 7B	32	4,096	32	11,008	3.9M

Here n denotes the number of input tokens. $\mathbf{p}_j^T(x^{i+1})$ denotes the token distribution over the vocabulary at the i^{th} step predicted by the teacher model, while $\mathbf{p}^S(x^{i+1})$ is the corresponding predicted distribution of the student model.

QAT utilizing knowledge distillation has been shown to be effective in various studies [21, 22, 23, 24, 7]. However, unlike these works, our training process exclusively uses the autoregressive distillation loss without adding any other losses to maintain simplicity. Our experiments verified that using only the distillation loss yields better results than the vanilla one-hot label based autoregressive loss while maintaining methodological simplicity when working with fully binarized LLMs.

Since the $\text{sign}(\cdot)$ is non-differentiable at zero, it causes the gradient chain to break during backpropagation, preventing optimization of the model parameters. Therefore, we use the Straight-Through Estimator (STE) method [15] during backpropagation, where the gradient of the output of the non-differentiable function is used as an estimate for the gradient of the input, thus allowing the gradient to be effectively propagated. This estimation can be formulated as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^f} = \frac{\partial \mathcal{L}}{\partial \mathbf{W}^b} \quad (9)$$

4 Experiments

In our experiment, we follow the W1A16 setup [18, 7], quantizing only the parameters to 1-bit while keeping the activation values at 16-bit. We train FBI-LLMs with sizes of 130M, 1.3B, and 7B, testing their performance across multiple tasks.

4.1 Setup

Dataset. We train FBI-LLMs with the Amber dataset [25]. Amber dataset is a mixture of Refined-Web [26], StarCoder [27], and RedPajama-v1 [28] and contains the total 1.26 trillion tokens. It divides the data into 360 chunks, with each chunk containing an average of 3.5 billion tokens².

Training details. Our models used for experiments adopt a similar structure as LLaMA2 [3]. For the specific hyper-parameters settings of FBI-LLMs of different sizes, refer to Table 1. The maximum sequence length is set to 2048. The optimizer is Adam with $\beta_1 = 0.9$, $\beta_2 = 0.98$. The initial learning rate for all model sizes is set at $3e - 4$, following a cosine learning rate schedule that decreases to a final rate of $3e - 5$ as it is warmed up over 2,000 steps. We use gradient clipping at 1.0. We use LLaMA2-7B as the teacher model for all size FBI-LLMs to calculate autoregressive distillation loss. We train models with 64 NVIDIA A100 GPUs in total and maintain BF16 precision while training. Please refer to Appendix C for more details.

Baselines. We compare our work with prior binarized LLMs Bi-LLM [18], OneBit [7], and BitNet [19]. We also include the BitNet b1.58 [1], which is a ternary quantization LLM, as our baseline model for comparison³. We further include results from open-sourced full-precision models of various sizes, such as OPT [29], LLaMA [3, 4], and TinyLLaMA [30], as references.

Evaluation Metrics. We evaluate the models based on their zero-shot performance in some downstream tasks, including BoolQ [31], PIQA [32], HellaSwag [33], Winogrande [34], ARC [35], and OpenbookQA [36]. We also use perplexity as the evaluation metric. Perplexity measures how well a probability model predicts a token, quantitatively measuring the model’s generation power.

²As shown in Fig. 3, in our experiment, about 10% of the training data chunks have already achieved competitive performance. Further training is naturally expected to yield even higher accuracy.

³As the original BitNet b1.58 [1] has not open-sourced their model, we use a third-party open-sourced one <https://huggingface.co/1bitLLM/> to evaluate certain indicators for the comparison. This model achieves slightly better results than those reported in the original paper.

Table 2: Performance on downstream tasks and perplexity. Here, BW means bit-width, which refers to the average number of bits occupied by each parameter. HS, WG, and OBQA are abbreviations for HellaSwag, Winogrande, and OpenbookQA, respectively. We divide the table into three blocks based on model size. In each block, the bold values represent the best values among the non-high-precision models, while the values with an underline represent the best values among the high-precision models.

Model	Size	BW	Zero-shot Accuracy $\uparrow$								Perplexity $\downarrow$		
			BoolQ	PIQA	HS	WG	ARC-e	ARC-c	OBQA	Ave.	Wiki2	PTB	C4
BitNet b1.58 [1]	700M	1.59	58.2	68.1	35.1	55.2	51.8	21.4	20.0	44.3	17.1	72.1	17.5
FBI-LLM (Ours)	130M	1.01	62.1	59.3	28.7	51.0	34.9	20.5	26.4	40.4	28.2	136.6	26.9
TinyLLaMA [30]	1.1B	16	<u>57.8</u>	<u>73.3</u>	<u>59.2</u>	59.1	<u>55.3</u>	<u>30.1</u>	<u>36.0</u>	<u>53.0</u>	<u>7.8</u>	30.5	9.9
OPT [29]	1.3B	16	<u>57.8</u>	72.5	53.7	<u>59.5</u>	51.0	29.5	33.4	51.1	14.6	<u>20.3</u>	16.1
OneBit-OPT [7]	1.3B	1.02	<u>59.5</u>	62.6	34.3	<u>51.1</u>	41.3	24.1	-	-	25.4	-	23.0
BitNet b1.58 [1]	1.3B	1.59	56.7	68.8	37.7	55.8	54.9	24.2	19.6	45.4	24.1	145.1	21.8
FBI-LLM (Ours)	1.3B	1.01	60.3	69.0	42.3	54.0	43.6	25.3	29.6	46.3	12.6	39.3	13.8
OPT [29]	7B	16	66.1	76.5	67.2	65.4	60.0	34.7	37.4	58.2	10.9	<u>15.8</u>	12.7
LLaMA [3]	7B	16	75.1	<u>79.2</u>	<u>76.2</u>	<u>69.9</u>	72.9	44.9	<u>44.4</u>	<u>66.0</u>	5.7	41.2	<u>7.3</u>
LLaMA2 [4]	7B	16	<u>77.7</u>	79.1	76.0	69.1	<u>74.6</u>	<u>46.2</u>	<u>44.2</u>	<u>66.7</u>	<u>5.5</u>	37.9	<u>7.3</u>
OneBit-LLaMA2 [7]	7B	-	63.1	68.1	52.6	58.4	41.6	29.6	-	-	9.7	-	11.1
BitNet [19]	7B	-	-	-	38.9	51.4	-	-	-	-	-	-	-
BiLLM-OPT [18]	7B	1.11	62.2	58.6	31.9	51.5	34.1	23.9	29.0	41.6	35.4	73.6	43.2
BiLLM-LLaMA [18]	7B	1.08	62.7	61.2	36.8	51.1	36.0	25.7	31.8	43.6	35.0	421.3	39.6
BiLLM-LLaMA2 [18]	7B	1.08	61.8	60.6	34.8	52.4	36.2	24.4	33.2	43.3	32.5	3877.4	40.5
FBI-LLM (Ours)	7B	1.01	61.5	72.6	57.7	58.9	53.0	29.9	36.8	52.9	9.1	29.6	10.5

4.2 Main Results

Table 2 presents the main results comparing our FBI-LLMs to various state-of-the-art baseline models. We also report the average bit-width occupied by model parameters, excluding the embedding layer and the head, for different models. Details on the calculation process can be found in Appendix B. Our FBI-LLMs maintain the lowest average bit-width across different model sizes while demonstrating remarkable performance. We provide zero-shot accuracy, which is a foundation for understanding how well a model can perform without additional task-specific information. This metric is commonly used to assess the model’s initial capabilities and aligns with certain benchmarking tasks aimed at measuring the pre-trained LLM’s general comprehension and knowledge-reserving capabilities across diverse downstream tasks without additional few-shot information.

Since there is no binary baseline for the 130M size, we compare our 130M model with the BitNet b1.58 at the 700M scale. Despite the fivefold difference in model size and significant variations in quantization degree, our model still outperforms BitNet b1.58 in BoolQA and OpenbookQA. For the 1.3B-scale binary models, our FBI-LLM achieves the best performance in most downstream tasks and perplexity, even matching or exceeding the capacity of some 7B-scale binary models like BiLLM-LLaMA2-7B. Compared to the full-precision models of a similar scale, the proposed FBI-LLM 1.3B can achieve up to 87% of their performance in downstream tasks. In the 7B scale, our model significantly outperformed nearly all baselines.

Furthermore, limited by computational resources, the current results for FBI-LLM 7B are not final. We only use 8.6% (31 chunks) of the Amber dataset. Fig. 3 illustrates the changes in downstream task accuracy and perplexity during the training process of FBI-LLM-7B. It is clear that, as of the current training progress, the performance of FBI-LLM-7B will be improved consistently, and further training could yield better results.

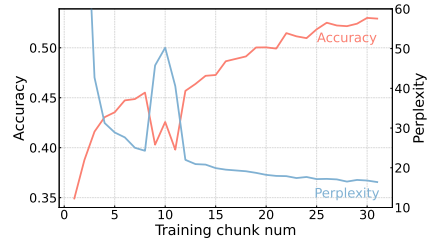


Figure 3: Changes in average perplexity and downstream task accuracy during the training of FBI-LLM 7B. The horizontal axis represents the number of Amber data blocks used for training.

4.3 Effectiveness of Autoregressive Distillation

To demonstrate the effectiveness of using only autoregressive distillation as the training objective, we train two models: one using solely the autoregressive distillation loss and the other using only the standard autoregressive loss. All other training procedures are identical to those used for FBI-LLM. We evaluate the performance of these models on downstream tasks and perplexity, as shown in Fig. 4. The evaluation tasks and datasets are the same as those listed in Table 2. For clarity, we present only the average accuracy across different tasks and the

average perplexity across different datasets here. Detailed performance for each task is provided in Appendix D.

It can be observed that throughout the training process, models trained with autoregressive distillation objective consistently outperform those trained with the standard autoregressive scheme in downstream tasks and perplexity. This indicates that using autoregressive distillation objective is more effective in training binarized LLMs. The utilized soft labels from the output probabilities of the teacher model contain more information than hard labels (i.e., the vocabulary labels). They provide a distribution over all possible vocabulary, indicating not just the target word but also the relative confidence in other possible words. This richer information helps the target model learn nuanced patterns and relationships in the data that are captured by the strong teacher model. Since our target model is learning solely from a smoothed version of the ground truth, it is less likely to overfit to the noise or specific details in the training data that may not generalize well to new data. Therefore, retaining only autoregressive distillation as the training objective ensures the simplicity and effectiveness of the entire training workflow.

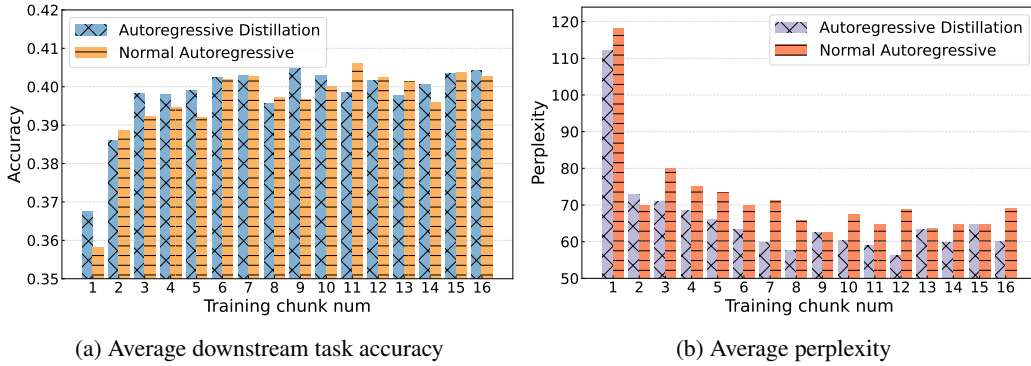


Figure 4: The model performance for different training loss.

5 Analysis

In this section, we analyze 1) the better choice of training from scratch or continuing training from pretrained LLM for binarized LLMs. 2) training instability and our solution. 3) storage efficiency of our models. 4) generation case demonstrations.

5.1 Training from Scratch or Continue Training from Pretrained LLM?

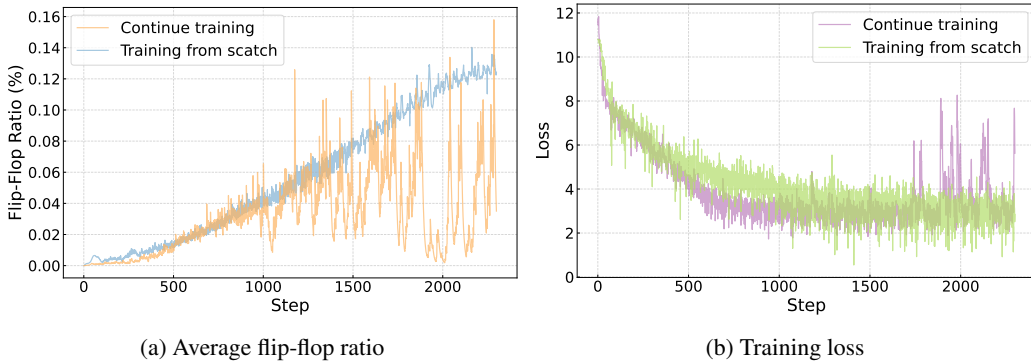


Figure 5: The flip-flop ratio and loss for different training procedures.

Intuitively, continuing training from a pretrained LLM to obtain a binarized model can inherit knowledge from the original model, potentially achieving better results than training from scratch. To evaluate this hypothesis, we conduct analytical experiments to record and compare the behaviors of models under both training procedures.

To quantify and examine the model behaviors using pretrained parameters or training from scratch, as well as their stability and initialization dependency of them, we apply the flip-flop (FF) ratio [37]. The FF ratio measures optimization stability behavior, which is defined as follows:

$$\mathbf{C}_{\mathbf{FF}} = \frac{|\text{Sign}(w_{t+1}^b) - \text{Sign}(w_t^b)|_{\text{abs}}}{2}, \quad (10)$$

where $\mathbf{C}_{\mathbf{FF}}$ is an indicator that shows whether a binary weight changes its sign after the update at iteration t , and $|\cdot|_{\text{abs}}$ is the absolute operation.

$$\mathbf{FF}_{\text{ratio}} = \frac{\sum_{l=1}^L \sum_{w^b \in \mathbf{W}_l^b} \mathbf{C}_{\mathbf{FF}}}{N_{\text{total}}}, \quad (11)$$

where N_{total} represents the total number of parameters in a model with L layers. $\mathbf{FF}_{\text{ratio}}$ denotes the flip-flop ratio, which is the percentage of parameters that change their signs.

In this experiment, we select TinyLLaMA as the base LLM and initialize the binarized LLM parameters using either pretrained values or random assignments. We maintain consistency in all other training details with the FBI-LLM. In addition to the $\mathbf{FF}_{\text{ratio}}$, we monitor training losses and gradient norms prior to gradient clipping across both training procedures.

From Fig. 5a, it is observed that during the beginning of training, the trend of $\mathbf{FF}_{\text{ratio}}$ remains largely consistent between the two procedures. Throughout the training process, the $\mathbf{FF}_{\text{ratio}}$ for both methods are on similar scales and the values are relatively small. It indicates that the two different parameter initialization approach do not significantly differ in their effects on binarization optimization process. Fig. 5b shows changes in training loss for two training procedure. At the initial stage of training, the training losses for the two approaches are essentially the same, shows that the model’s training loss is not notably influenced by the initialization approach. Although the loss of training from scratch is slightly higher than that of continuing training after a period of time, after some more time, the loss of training from scratch once again matches or even becomes less than that of continuing training, and the trend of loss change in training from scratch is more stable. Significantly, at approximately the 1000th step, as illustrated in Fig. 5a, the $\mathbf{FF}_{\text{ratio}}$ begins to show notable fluctuations when continuing training from a pretrained LLM. Similarly, at around the 1700th step shown in Fig. 5b, the training loss experiences similar issues. Additionally, Fig. 6 highlights more pronounced changes in the gradient norms during the continuing training.

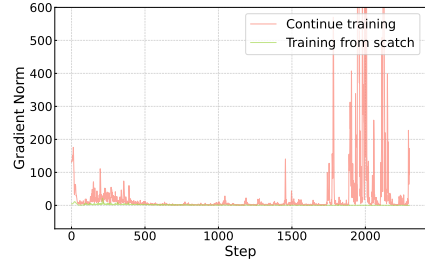


Figure 6: Gradient norm curves from different training procedures.

These findings challenge our initial hypothesis that starting with a pretrained LLM would endow the binarized model with inherited knowledge, thus enhancing performance. Instead, they imply that binarization through training is not sensitive to the way of parameter initialization. Furthermore, we speculate that binarized and full-precision LLMs employ different parameter combinations and configurations to encode semantics, which results in substantial divergences in their parameter space pattern. To adapt this pattern, the optimization process for binarization by continuing training from a pretrained LLM might necessitate more profound parameter adjustments. This could partly explain why it is more unstable compared to training from scratch during the training.

5.2 Training Instability

Both binary and full-precision LLM training have been found to exhibit unstable training behaviors [19, 7, 38]. Our FBI-LLM exhibits similar issues, specifically manifesting as sudden spikes in training loss when training 1.3B and 7B FBI-LLMs, which sometimes fail to converge after that. We adopt the solution similar to PaLM [38]: if the loss no longer tends to converge, the model reverts to the previous checkpoint and skips the data chunk that triggered the unstable loss to continue training. The model no longer encounters issues at the same training steps using this approach. We observe that pretraining the 7B FBI model from scratch has approximately a 6% probability of causing loss spikes. For the 1.3B model, training is more unstable due to the lower capability, with about a 15% probability of loss spikes. This is consistent with the pretraining behavior seen in real-valued LLMs while the probability of spiking is significantly higher, which may be related to the limited expressive capability of binary parameters. To handle this, we skip any data blocks where loss spikes occur.

5.3 Storage Efficiency

Table 3: Compression and extra parameters ratio.

Model	Model Size	Storage Size	Compression Ratio	Extra Parameter Ratio
LLaMA	130M	0.25GB	59.26%	0.119%
FBI-LLM	130M	0.10GB		
LLaMA	1.3B	2.54GB	84.67%	0.063%
FBI-LLM	1.3B	0.39GB		
LLaMA	7B	12.55GB	90.07%	0.034%
FBI-LLM	7B	0.39GB		

Table 3 presents the theoretical storage space required by FBI-LLMs of various sizes compared to the full-precision LLaMA with the same structure. It also details the proportion of additional parameters (α and β) introduced by FBI-LLM. The comparison in the table demonstrates that FBI-LLM can achieve a high compression ratio, significantly reducing the storage burden of LLMs. Although the extra parameters for scaling and shifting introduced by FBI-LLM need to be retained in full precision, their proportion is minimal, rendering their impact on storage negligible.

5.4 Generation Cases

As illustrated in Fig. 7, although the generation quality of FBI-LLMs does not fully match that of full-precision LLMs, FBI-LLMs can still generate fluent and meaningful content. Compared to BitNet b1.58, which has a higher parameter bit-width, FBI-LLMs demonstrate a better understanding of prompts and include more knowledge in some generated examples. This indicates that FBI-LLMs possess strong generative capabilities and contain sufficient knowledge. Furthermore, FBI-LLMs demonstrate the potential to scale up further, reaching new levels of intelligence while being more hardware-friendly for deployment.

Prompt: Create a Twitter post to promote your new product: Alisha's All Natural Jam	
(1.3B) OPT:	I'm having a jam sale today! Come check it out! I'm offering 4 different jams for \$5 each. This is your last chance to get them before I sell out!\n\nIf you are a follower of mine on Twitter or Facebook, you will be notified when the sale is live.\n\nI'm having a jam sale today! Come check it out! I'm offering 4 different jams for \$5 each. This is your last chance to get them before I sell out!\n\nIf you are a follower of mine on Twitter or Facebook, you will be notified when the sale is live.
(1.1B) TinyLLaMA:	The New Product You Want to Sell\nAlisha's All Natural Jam The New Product You Want to Sell\nAlisha's All Natural Jam is a handcrafted jam using only the finest ingredients in an easy to use 1-cup package that tastes like a 12-cup jar. It's perfect for a gift, a snack, or you can use it for a delicious dessert at home.\nAlisha's All Natural Jam is made using real fruit, no preservatives, no added sugar, no GMOs, and is free of allergens. It's sweetened with coconut sugar, and gluten-free. It's a great addition to any breakfast, snack, or dessert.
(1.3B) BitNet 1.58bit:	Alisha's All Natural Jam and Conditioner - 4 oz.
(1.3B) FBI-LLM:	Alisha's All Natural Jam is a delicious jam that is made with only the finest ingredients. It has a subtle sweetness that pairs well with all types of fruit. It's perfect for juicing, sourcing, or making jam from scratch. It's a healthy snack you can have all year round, and it's a perfect way to incorporate fresh fruits and veggies into your meals.\nAlisha's All Natural Jam is a great addition to your salad dressing, but can also be used as a dip for vegetables. Its light texture makes it perfect for juicing!\nAlisha's All Natural Jam is a delicious jam that is made with only the finest ingredients.
Prompt: Correct the grammatical errors in the sentence: She come to the store for supplies.	
(7B) LLaMa2:	The grammatical error in the sentence is "She come ". It should be "She came ".
(3B) BitNet 1.58bit:	She can not go to the store for supplies.\nShe comes to the store for supplies.\nShe can not go to the store for supplies.\nShe comes to the store for supplies. She can not go to ...
(7B) FBI-LLM:	Correct: She comes to the store for supplies.
Prompt: Design a logo for a web development company.	
(7B) LLaMa2:	The logo should be simple and elegant. The color scheme should be red, white, and blue.
(3B) BitNet 1.58bit:	I was asked to create a logo for a web development company. From the brief I came up with 2 options.\n- A stylised 'H' and 'D' which could be used in the logo and text.\n- A more modern 'H' and 'D' which would be more suitable for a web development company.\nI chose option 2 as it would be more suitable for a web development company.\nThe final logo I came up with is below.\nI'm happy to have created this logo for a web development company.
(7B) FBI-LLM:	The client is a web development company with a large and loyal customer base. They require a logo, which incorporates their company's branding and colours, and is available on print and on the web.\nThe logo will need to adhere to the company's brand, colour palette and style guide.\nThe logo should be simple, clean and consistent, and should not be too busy.\nThe logo will be used on the company website, on business cards and on all promotional material.\nThe client has a strong preference for a logo that can be easily recognisable and that can be used at all stages of the development cycle.\nLogo design: Please provide a logo design.

Figure 7: Generation cases. We compare the outputs of the full-precision model, BitNet b1.58, and our FBI-LLM when given the same prompts.

6 Conclusion

We have proposed a learning framework using autoregressive distillation for 1-bit weight binarization of LLMs from scratch. Extensive experiments on models of various sizes of 130M, 1.3B, and 7B demonstrate that FBI-LLM outperforms strong baselines and strikes a good balance between model size and performance. We also analyze the capabilities, properties, and potential of these extremely low-bit models, providing guidance for future research.

Limitations. Our proposed binarization framework significantly reduces the memory and computation consumptions of LLMs, providing potential for their efficient deployment. However, there are several limitations of our models. Firstly, our 1-bit binarization inevitably incurs a performance loss compared to the original full-precision model. Additionally, the training process, which includes knowledge distillation, brings additional computational costs. Moreover, due to the unique nature of binarization, current hardware makes it difficult to directly support binarized LLMs to achieve real speedup. We also have not yet considered intermediate activation binarization which is the same as previous studies. Finally, potential ethical issues of pretrained LLMs, such as harmful biases, privacy concerns, and the spread of disinformation, are likely to persist after binarization in our LLMs.

References

- [1] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit llms: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024.
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [3] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [4] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [5] Yuzhang Shang, Zhihang Yuan, and Zhen Dong. Pb-llm: Partially binarized large language models. In *The Twelfth International Conference on Learning Representations*, 2023.
- [6] Elias Frantar and Dan Alistarh. Optimal brain compression: A framework for accurate post-training quantization and pruning. *ArXiv*, abs/2208.11580, 2022.
- [7] Yuzhuang Xu, Xu Han, Zonghan Yang, Shuo Wang, Qingfu Zhu, Zhiyuan Liu, Weidong Liu, and Wanxiang Che. Onebit: Towards extremely low-bit large language models. *arXiv preprint arXiv:2402.11295*, 2024.
- [8] Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. Binaryconnect: Training deep neural networks with binary weights during propagations. *Advances in neural information processing systems*, 28, 2015.
- [9] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Binarized neural networks. *Advances in neural information processing systems*, 29, 2016.
- [10] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. Xnor-net: Imagenet classification using binary convolutional neural networks. In *European conference on computer vision*, pages 525–542. Springer, 2016.
- [11] Shuchang Zhou, Yuxin Wu, Zekun Ni, Xinyu Zhou, He Wen, and Yuheng Zou. Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients. *arXiv preprint arXiv:1606.06160*, 2016.
- [12] Asit Mishra, Eriko Nurvitadhi, Jeffrey J Cook, and Debbie Marr. Wrpn: Wide reduced-precision networks. *arXiv preprint arXiv:1709.01134*, 2017.
- [13] Xiaofan Lin, Cong Zhao, and Wei Pan. Towards accurate binary convolutional neural networks. *arXiv preprint arXiv:1711.11294*, 2017.

- [14] Antonio Polino, Razvan Pascanu, and Dan Alistarh. Model compression via distillation and quantization. *arXiv preprint arXiv:1802.05668*, 2018.
- [15] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- [16] Zechun Liu, Zhiqiang Shen, Marios Savvides, and Kwang-Ting Cheng. Reactnet: Towards precise binary neural network with generalized activation functions. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XIV 16*, pages 143–159. Springer, 2020.
- [17] Sajad Darabi, Mouloud Belbahri, Matthieu Courbariaux, and Vahid Partovi Nia. BNN+: Improved binary network training, 2019.
- [18] Wei Huang, Yangdong Liu, Haotong Qin, Ying Li, Shiming Zhang, Xianglong Liu, Michele Magno, and Xiaojuan Qi. Billm: Pushing the limit of post-training quantization for llms. *arXiv preprint arXiv:2402.04291*, 2024.
- [19] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. Bitnet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*, 2023.
- [20] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.
- [21] Jangho Kim, Yash Bhalgat, Jinwon Lee, Chirag Patel, and Nojun Kwak. Qkd: Quantization-aware knowledge distillation. *arXiv preprint arXiv:1911.12491*, 2019.
- [22] Yoonho Boo, Sungho Shin, Jungwook Choi, and Wonyong Sung. Stochastic precision ensemble: self-knowledge distillation for quantized deep neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 6794–6802, 2021.
- [23] Cuong Pham, Tuan Hoang, and Thanh-Toan Do. Collaborative multi-teacher knowledge distillation for learning low bit-width deep neural networks. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 6435–6443, 2023.
- [24] Kartikeya Bhardwaj, Nilesh Prasad Pandey, Sweta Priyadarshi, Kyunggeun Lee, Jun Ma, and Harris Teague. Oh! we freeze: Improving quantized knowledge distillation via signal propagation analysis for large language models. *arXiv preprint arXiv:2403.18159*, 2024.
- [25] Zhengzhong Liu, Aurick Qiao, Willie Neiswanger, Hongyi Wang, Bowen Tan, Tianhua Tao, Junbo Li, Yuqi Wang, Suqi Sun, Omkar Pangarkar, et al. Llm360: Towards fully transparent open-source llms. *arXiv preprint arXiv:2312.06550*, 2023.
- [26] Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Alessandro Cappelli, Hamza Alobeidli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. The refinedweb dataset for falcon llm: outperforming curated corpora with web data, and web data only. *arXiv preprint arXiv:2306.01116*, 2023.
- [27] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.
- [28] Together Computer. Redpajama: An open source recipe to reproduce llama training dataset, 2023.
- [29] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.
- [30] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024.
- [31] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. Boolq: Exploring the surprising difficulty of natural yes/no questions. *ArXiv*, abs/1905.10044, 2019.
- [32] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439, 2020.

- [33] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Annual Meeting of the Association for Computational Linguistics*, 2019.
- [34] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- [35] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *ArXiv*, abs/1803.05457, 2018.
- [36] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Conference on Empirical Methods in Natural Language Processing*, 2018.
- [37] Zechun Liu, Zhiqiang Shen, Shichao Li, Koen Helwegen, Dong Huang, and Kwang-Ting Cheng. How do adam and training strategies help bnns optimization. In *International conference on machine learning*, pages 6936–6946. PMLR, 2021.
- [38] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.

Appendix

A Broader Impacts

Our proposed fully binarized large language models (FBI-LLM) require less computational power and memory in training and inference, making advanced AI technology accessible to organizations and individuals with limited resources. With reduced hardware requirements, smaller businesses, educational institutions, and non-profit organizations can implement LLMs, democratizing access to cutting-edge AI. Binarized models are more energy-efficient, which can significantly lower the carbon footprint associated with running large-scale AI applications. However, even binarized LLMs can still inherit and exist biases present in training data, leading to unfair outcomes in applications like hiring, law enforcement, and lending.

B Average Bit-width of Binarized LLM

This section explains how to calculate the average bit-width of a binarized LLM. Since the embedding layer and head have a large number of parameters and are not binarized, we do not consider them when calculating the average bit-width. Consider a module containing an RMSNorm and a linear layer with a parameter matrix $A \in \mathbb{R}^{n \times n}$, which in total has $n + n^2$ parameters. Using the FBI-LLM binarization process as an example, after binarization, this module gains additional learnable scale parameters α and β , totaling $2n$ parameters. During inference, the parameters of RMSNorm, α , and β remain at 16 bits, while A is quantized to 1 bit. Therefore, the average bit-width of this module can be calculated as follows:

$$\text{Average Bit-width} = \frac{1 \times n^2 + 16 \times 3n}{3n + n^2} \quad (12)$$

C Model Configuration and Training Details

In this section, we list the model configurations and training details for three scales of FBI-LLM models we trained in Table 4.

Table 4: The configuration and training details for FBI-LLM.

	FBI-LLM 130M	FBI-LLM 1.3B	FBI-LLM 7B
hidden size	768	2,048	4,096
intermediate size	2,048	5,632	11,008
max sequence length	2,048	2,048	2,048
# attention heads	12	32	32
# hidden layers	12	24	32
# key value heads	12	32	32
initializer range	0.02	0.02	0.02
vocabulary size	32,000	32,000	32,000
learning rate	3e-4	3e-4	3e-4
batch size (token)	2M	2.4M	3.9M
teacher model	LLaMA2-7B	LLaMA2-7B	LLaMA2-7B
# GPUs (A100 80G)	16	16	32
GPU hours for each data chunk	130h	189h	729h
training speed (tokens/s/GPU)	7,800	5,300	1,200

D Detail Experiment Results

We list the detailed experiment results about the effectiveness of autoregressive distillation in Section 4.3 in Table 5.

Table 5: Performance on down stream tasks and perplexity for different training objectives. Here, NA means normal autoregressive training objective and AD means autoregressive distillation training objective. Based on the number of training chunk, the table is divided into several blocks. In each block, the values with an underline represent the best value.

Training Chunk	Loss Type	Zero-shot Accuracy $\uparrow$								Perplexity $\downarrow$		
		BoolQ	PIQA	HS	WG	ARC-e	ARC-c	OBQA	Ave.	Wiki2	PTB	C4
1	NA	42.2	53.9	26.2	<u>52.3</u>	<u>29.3</u>	21.3	<u>25.4</u>	35.8	85.9	204.4	63.9
	AD	<u>50.1</u>	<u>54.1</u>	<u>26.6</u>	51.0	29.2	<u>21.9</u>	24.4	<u>36.8</u>	<u>81.2</u>	193.4	61.9
2	NA	60.8	<u>56.7</u>	27.0	<u>52.2</u>	<u>31.9</u>	20.2	<u>23.2</u>	<u>38.9</u>	<u>42.9</u>	<u>128.8</u>	37.5
	AD	<u>61.5</u>	56.1	<u>27.3</u>	50.8	31.1	<u>20.6</u>	22.8	38.6	43.4	137.9	<u>37.3</u>
3	NA	62.0	57.1	27.5	<u>51.3</u>	31.5	21.2	24.0	39.2	36.7	170.2	<u>32.8</u>
	AD	<u>62.2</u>	<u>58.1</u>	<u>27.7</u>	51.1	<u>33.0</u>	<u>21.3</u>	<u>25.4</u>	<u>39.8</u>	<u>34.9</u>	<u>145.2</u>	32.9
4	NA	61.0	<u>57.9</u>	27.6	<u>52.7</u>	32.6	21.0	23.4	39.5	34.3	159.6	31.3
	AD	<u>62.0</u>	57.7	<u>27.7</u>	50.1	<u>33.0</u>	<u>21.2</u>	<u>26.8</u>	<u>39.8</u>	<u>32.9</u>	<u>142.1</u>	<u>31.0</u>
5	NA	61.9	57.6	27.7	49.6	32.7	21.2	23.6	39.2	32.3	157.9	29.7
	AD	<u>62.2</u>	<u>58.4</u>	<u>28.0</u>	<u>50.8</u>	<u>32.9</u>	<u>21.3</u>	<u>25.8</u>	<u>39.9</u>	<u>31.7</u>	<u>137.0</u>	29.4
6	NA	62.1	<u>59.5</u>	<u>27.9</u>	<u>53.1</u>	32.9	21.8	24.0	40.2	32.4	147.4	29.6
	AD	<u>62.2</u>	59.2	27.7	49.9	33.9	<u>22.9</u>	<u>26.0</u>	<u>40.2</u>	31.3	129.0	29.4
7	NA	62.0	59.5	27.8	<u>52.4</u>	33.8	20.9	<u>25.6</u>	40.3	<u>30.0</u>	155.1	<u>28.5</u>
	AD	<u>62.1</u>	<u>59.5</u>	<u>28.3</u>	51.2	<u>33.8</u>	<u>21.8</u>	25.4	<u>40.3</u>	31.0	119.3	28.8
8	NA	61.6	58.8	<u>28.3</u>	<u>51.0</u>	<u>33.6</u>	<u>21.3</u>	23.4	<u>39.7</u>	<u>29.5</u>	140.0	<u>28.0</u>
	AD	<u>62.2</u>	<u>59.0</u>	28.1	50.8	32.3	20.6	24.0	39.6	30.1	113.8	28.9
9	NA	60.9	58.9	28.2	51.7	<u>34.6</u>	20.0	23.4	39.7	<u>29.7</u>	<u>129.3</u>	28.4
	AD	<u>62.2</u>	<u>60.1</u>	<u>28.2</u>	51.8	33.5	<u>21.1</u>	26.6	<u>40.5</u>	29.9	129.8	<u>27.9</u>
10	NA	61.8	<u>59.5</u>	27.9	50.3	34.0	20.8	25.8	40.0	29.5	144.7	28.1
	AD	<u>62.2</u>	58.5	28.0	51.4	<u>34.2</u>	<u>21.6</u>	26.2	<u>40.3</u>	30.1	122.8	28.1
11	NA	62.1	<u>59.9</u>	<u>28.4</u>	<u>52.7</u>	<u>34.1</u>	<u>21.6</u>	<u>25.4</u>	<u>40.6</u>	<u>28.8</u>	138.2	<u>27.5</u>
	AD	<u>62.2</u>	59.0	28.0	49.9	34.0	20.7	25.2	39.9	29.0	119.9	27.6
12	NA	<u>62.2</u>	59.1	27.9	51.9	<u>34.6</u>	21.2	<u>24.8</u>	40.2	29.1	150.3	<u>27.2</u>
	AD	62.1	<u>59.2</u>	<u>28.1</u>	<u>52.3</u>	34.0	21.2	24.2	40.2	28.6	113.0	27.3
13	NA	<u>62.2</u>	58.6	27.9	49.6	<u>34.6</u>	<u>22.6</u>	<u>25.4</u>	<u>40.1</u>	28.7	135.0	<u>27.0</u>
	AD	62.1	<u>58.8</u>	<u>28.2</u>	49.4	33.5	21.6	25.0	39.8	<u>28.4</u>	<u>134.0</u>	27.3
14	NA	<u>62.2</u>	58.3	<u>28.4</u>	49.4	<u>34.3</u>	20.7	23.8	39.6	28.6	138.5	26.9
	AD	61.5	<u>58.5</u>	28.3	51.0	33.9	<u>21.2</u>	<u>26.0</u>	<u>40.1</u>	<u>28.3</u>	<u>123.6</u>	27.3
15	NA	<u>62.2</u>	58.8	<u>28.1</u>	51.5	34.1	<u>21.3</u>	<u>26.4</u>	<u>40.4</u>	<u>28.0</u>	138.9	26.9
	AD	<u>62.2</u>	58.8	28.0	51.5	<u>34.8</u>	20.9	26.2	<u>40.4</u>	28.5	<u>138.3</u>	27.4
16	NA	61.3	58.4	<u>28.3</u>	51.7	<u>34.6</u>	21.7	25.8	40.3	28.6	151.9	27.0
	AD	<u>62.1</u>	<u>60.2</u>	28.1	<u>51.9</u>	33.2	<u>22.3</u>	25.2	<u>40.4</u>	<u>28.2</u>	<u>124.6</u>	27.2